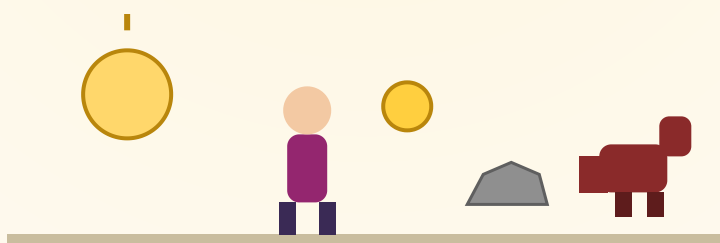


শিশুদের জন্য সহজ ব্যাখ্যা

ষ্টোন রানার

STONE RUNNER



এই বইটা তোমাকে দেখাবে, খেলাটার কোডের ভিতরে কী
কী "ফাংশন" (কাজের নাম) আছে, আর কোন ফাংশন কাকে
ডেকে খেলাটা চালায়।

ছবি আর সহজ গল্পের মাধ্যমে বোঝানো হয়েছে, যাতে খুব ছোট বন্ধুরাও
বুঝতে পারে।

Dino_Sprint / Stone Runner প্রজেক্ট • কোড থেকে তৈরি ডকুমেন্টেশন

এই বইয়ে যা যা আছে

১. কিছু সহজ শব্দ আগে শিখে নিই (শব্দকোষ)
২. খেলাটা আসলে কী?
৩. পুরো কোডটা কীভাবে সাজানো আছে
৪. খেলা শুরু হওয়ার সময় কী কী ঘটে
৫. প্রতি মুহুর্তে (প্রতি ফ্রেমে) কী ঘটে
৬. main.cpp — প্রধান নিয়ন্ত্রকের সব ফাংশন
৭. DreamWorld.cpp — দিনের দৃশ্যের সব ফাংশন
৮. DreamHell.cpp — মরুভূমির দৃশ্যের সব ফাংশন
৯. dragon.cpp — ডাইনোসরের সব ফাংশন
১০. roshni_player.cpp — মেয়েটির সব ফাংশন
১১. বাড়তি ৩টি ফাইল (একা একা চলা প্র্যাকটিস ফাইল)
১২. শেষ কথা

কিছু সহজ শব্দ আগে শিখে নিই

এই বইয়ে কিছু ইংরেজি/কোডিং শব্দ বার বার আসবে। এগুলো একবার বুঝে নিলে, বাকি পুরো বই পড়া অনেক সহজ হয়ে যাবে।

ফাংশন (function) — কম্পিউটারকে দেওয়া একটা ছোট্ট কাজের নাম। যেমন `drawStone()` মানে "একটা পাথর আঁকো" — এই নামটা ডাকলেই কম্পিউটার পাথর আঁকে দেয়।

ডাকা / কল করা (call) — একটা ফাংশনের ভেতর থেকে আরেকটা ফাংশনের নাম লিখে তাকে কাজে লাগানো। যেমন একজন বড় আপু ছোট ভাইকে ডেকে বলে "যাও, এটা করে এসো।"

প্যারামিটার (parameter) — ফাংশনকে কাজ করার সময় যে তথ্যটুকু দিতে হয়। যেমন `drawCircle(cx, cy, r)` কে বলতে হয় বৃত্তটা কোথায় (cx, cy) আর কত বড় (r) আঁকতে হবে।

x, y কোঅর্ডিনেট — স্ক্রিনের মধ্যে একটা জিনিসের অবস্থান। x মানে বামে-ডানে কতদূর, y মানে নিচে-উপরে কতদূর।

static (গোপন) ফাংশন/চলক — এই ফাংশন বা মান শুধু নিজের ফাইলের ভেতরেই ব্যবহার হয়, বাইরের ফাইল থেকে ডাকা যায় না। যেন নিজের ঘরের গোপন খেলনা।

হেডার ফাইল (.h ফাইল) — একটা ফাইলের "দরজার তালিকা"। এখানে লেখা থাকে কোন কোন ফাংশন অন্য ফাইল থেকে ডাকা যাবে (public / বাইরে থেকে দেখা যায়)।

OpenGL / GLUT — একটা আঁকার সরঞ্জামবাক্স (drawing toolkit)। এটা দিয়ে কম্পিউটারের স্ক্রিনে রঙিন আকৃতি (চতুর্ভুজ, বৃত্ত, লেখা) আঁকা যায় এবং কীবোর্ড চাপলে কী হবে তা ঠিক করা যায়।

কলব্যাক ফাংশন (callback) — এমন একটা ফাংশন, যাকে আমরা নিজে ডাকি না; বরং GLUT নিজে থেকেই ঠিক সময়ে ডেকে দেয় (যেমন — একটা ছবি আঁকতে হলে, বা কীবোর্ড চাপলে)।

খেলাটা আসলে কী?

কোডের ভেতরে ঢোকার আগে, খেলাটা কীভাবে খেলা হয় সেটা জেনে নেওয়া যাক।

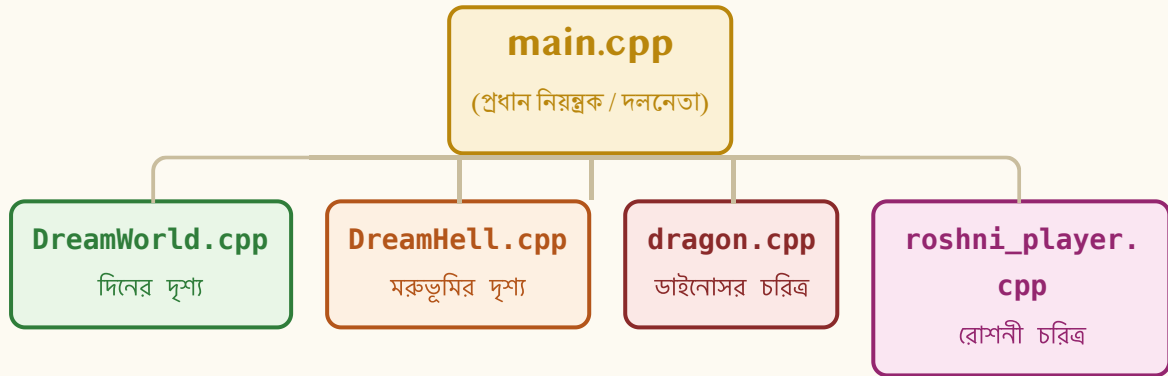
"স্টোন রানার" একটা দৌড়ের খেলা। একটা মেয়ে (রোশনী) সমানে দৌড়াতে থাকে। রাস্তায় পাথর (stone) থাকে — তাকে লাফিয়ে (SPACE বাটন চেপে) উপকাতে হয়। রাস্তায় সোনালি কয়েনও (coin) ভাসতে থাকে — কয়েনের গায়ে লাগলেই সেটা কুড়িয়ে নেওয়া হয় আর স্কোর বাড়ে। পেছন থেকে একটা ডাইনোসর (dragon/dino) তাড়া করে। যদি পাথরে ধাক্কা লেগে সব "লাইফ" (জীবন) শেষ হয়ে যায়, তাহলে ডাইনোসরটা দৌড়ে এসে মেয়েটাকে ধরে ফেলে আর খেলা শেষ হয়ে যায় (Game Over)। তখন ENTER চাপলে খেলা আবার নতুন করে শুরু করা যায়।

খেলা শুরু হলে একটা মেনু (menu) দেখা যায়, যেখানে **1** চাপলে দিনের বেলার দৃশ্য "DreamWorld" (ঘাস, সূর্য, পাহাড়) আর **2** চাপলে মরুভূমির দৃশ্য "DreamHell" (বালি, ক্যাকটাস) দেখা যায়। দুটো দৃশ্যই খেলার নিয়ম একই — শুধু পেছনের ছবি আলাদা। যেকোনো সময় **M** চাপলে আবার মেনুতে ফেরত যাওয়া যায়, আর **ESC** চাপলে খেলা বন্ধ হয়ে যায়।

পুরো কোডটা কীভাবে সাজানো আছে

একটা বড় কাজ ভাগ করে দিলে করা সহজ হয় — এই খেলার কোডও তাই কয়েকটা ফাইলে ভাগ করা।

main.cpp হলো এই খেলার "দলনেতা" বা হাব (hub)। এটা নিজে থেকে ছবি আঁকে না প্রতিটা জিনিসের — বরং ৪টা সাহায্যকারী ফাইলকে (মডিউল) ডেকে ডেকে কাজ করায়। প্রতিটা সাহায্যকারী ফাইলের নিজের একটা কাজ আছে:



প্রতিটা সাহায্যকারী ফাইলের একটা নিজস্ব **.h** (হেডার) ফাইলও আছে — **DreamWorld.h**, **DreamHell.h**, **dragon.h**, **roshni_player.h**। এগুলো হলো সেই ফাইলের "দরজার তালিকা": এখানে যে ফাংশনগুলোর নাম লেখা আছে, শুধু সেগুলোই **main.cpp** ডাকতে পারে। বাকি সব ফাংশন সেই ফাইলের ভেতরেই গোপন (static) থাকে, বাইরে থেকে দেখা যায় না।

একটা মজার তুলনা: **main.cpp** যেন একটা রান্নাঘরের প্রধান শেফ। সে নিজে সবজি কাটে না, রুটি বেলে না — বরং ৪ জন সহকারীকে ডেকে বলে "তুমি সবজি রান্না করো", "তুমি রুটি বানাও"। প্রতিটা সহকারীর কাজ আলাদা আলাদা, কিন্তু চূড়ান্ত খাবারের থালাটা (স্ক্রিনের ছবি) শেফ-ই সাজিয়ে দেয়।

এছাড়া প্রজেক্টে আরও ৩টা ফাইল আছে — **allInOne.cpp**, **roshni.cpp**, আর **referance.cpp**। এই তিনটা ফাইল **main.cpp**-এর সাথে যুক্ত (linked) না; প্রতিটার নিজের আলাদা **main()** ফাংশন আছে, মানে এরা একা একাই সম্পূর্ণ একটা প্রোগ্রাম হিসেবে চলতে পারে। এগুলো ছিল খেলাটা তৈরি করার সময়কার প্র্যাকটিস/রেফারেন্স ভার্সন। এই বইয়ের ১১ নম্বর অধ্যায়ে এগুলো নিয়ে আলাদা করে সংক্ষেপে বলা হয়েছে।

খেলা শুরু হওয়ার সময় কী কী ঘটে

কম্পিউটারে প্রোগ্রামটা চালু করলেই সবচেয়ে আগে একটাই ফাংশন কাজ শুরু করে: **main()**। এটাই পুরো প্রোগ্রামের প্রথম দরজা।

main()

প্রোগ্রামের একদম প্রথম দরজা



**glutInit /
glutInitWindowSize
/ glutCreateWindow**

GLUT-কে বলে দেয়, "একটা
৮০০x৪৫০ সাইজের জানালা
(window) খুলে দাও"



init()

পর্দার রঙ, রেন্ডিং আর
ক্যামেরার হিসাব ঠিক করে,
তারপর resetRun() ডাকে



**glutDisplayFunc /
glutKeyboardFunc /
glutTimerFunc**

GLUT-কে বলে দেয় — "ছবি
আঁকতে হলে display()
ডেকো, কী চাপলে
keyboard() ডেকো, প্রতি ১৬
মিলিসেকেন্ড পর update()
ডেকো"



glutMainLoop()

এখান থেকে প্রোগ্রামের নিয়ন্ত্রণ
GLUT-এর হাতে চলে যায় —
এটা কখনো থামে না, যতক্ষণ না
খেলা বন্ধ করা হয়

মজার তুলনা: `main()` যেন একজন আয়োজক, যে একটা অনুষ্ঠানের আগে সব ব্যবস্থা করে রাখে — মঞ্চ বানায় (window), শোনার/দেখার নিয়ম বলে দেয় (callback register করে) — তারপর বলে "শুরু করো!" (`glutMainLoop()`)। এরপর থেকে কী হবে তা নির্ভর করে দর্শক (তুমি) কী বোতাম চাপছ তার ওপর।

`glutMainLoop()` শুরু হওয়ার পর থেকে, নিচের ৩টা ফাংশনকে আমরা নিজে কখনো ডাকি না — GLUT নিজে থেকেই ঠিক সময়ে এগুলো ডাকে, তাই এদের বলে **কলব্যাক ফাংশন (callback)**:

display()

যখনই পর্দা নতুন করে আঁকতে
হয়

keyboard()

যখনই একটা কী চাপা হয়

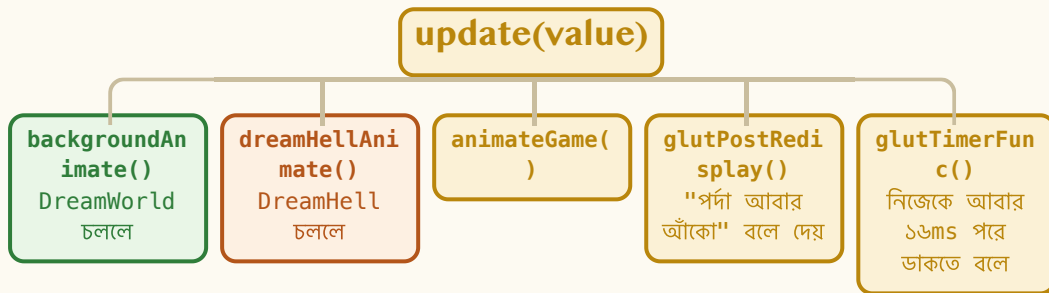
update()

প্রতি ১৬ মিলিসেকেন্ড পর পর,
ঘড়ির অ্যালার্মের মতো

প্রতি মুহূর্তে (প্রতি ফ্রেমে) কী ঘটে

খেলাটা ১ সেকেন্ডে প্রায় ৬০ বার (প্রতি ১৬ মিলিসেকেন্ডে একবার) নিজেকে আপডেট করে। এত দ্রুত হয় বলেই সবকিছু "নড়াচড়া করছে" মনে হয়, আসলে প্রতিটা ছবি একটু একটু বদলে যাচ্ছে।

৩ প্রতি ১৬ মিলিসেকেন্ডে একবার



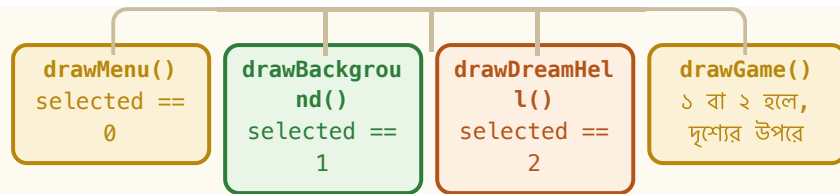
animateGame() হলো আসল খেলার ইঞ্জিন — এটাই স্কোর, লাইফ, চরিত্রদের নড়াচড়া সব একসাথে সামলায়:



একটা গুরুত্বপূর্ণ শেকল (**chain**): **checkStones()** যখন দেখে লাইফ (lives) শূন্য হয়ে গেছে, তখন সে **gameState**-কে ৩ বানিয়ে দেয় (মানে "ধরা পড়ে গেছে")। তার পরের ফ্রেম থেকে **animateGame()** প্রতিবার **dinoChase()** ডাকে, যেটা ডাইনোসরকে একটু একটু করে মেয়েটার কাছে নিয়ে আসে। ডাইনোসর একদম কাছে পৌঁছালে **dinoChase()** true ফেরত দেয়, আর তখন **gameState** হয়ে যায় ২ (মানে "Game Over")।

একই সময়ে, **display()** ফাংশনটা ঠিক করে পর্দায় কী দেখানো হবে:

display()



drawGame() নিজে আবার প্রতিটা খেলার বস্তু আঁকার জন্য অন্য ফাংশনদের ডাকে:



আর যখনই কীবোর্ডে কিছু চাপা হয়, GLUT **keyboard()**-কে ডাকে:



আর **gameKeyPress()** নিজেই আবার দুটো জায়গায় কাজের ভার দেয়: ENTER চাপলে `resetRun()` , আর SPACE চাপলে (`roshni_player` মডিউলের) `playerJumpStart()` ।

main.cpp – প্রধান নিয়ন্ত্রকের সব ফাংশন

২১টা ফাংশন আছে এই ফাইলে। নিচে এগুলোকে কাজ অনুযায়ী কয়েকটা দলে ভাগ করে দেখানো হলো।

খেয়াল করার মতো একটা জিনিস: `drawRect()`, `drawCircle()` আর `drawText()` — এই তিনটা "আঁকার হাতিয়ার" `main.cpp`-এ বানানো, কিন্তু `DreamWorld.cpp`, `DreamHell.cpp`, `dragon.cpp`, `roshni_player.cpp` — সবাই এগুলো ধার করে ব্যবহার করে। তারা প্রতিটা নিজের ফাইলের ওপরে `extern void drawRect(...)` লিখে বলে দেয়, "এই ফাংশনটা অন্য কোথাও বানানো আছে, আমাকে শুধু ব্যবহার করতে দাও।" এটা যেন পাশের বাড়ির কাছ থেকে একটা হাতুড়ি ধার নেওয়া, দরজার আনুষ্ঠানিক তালিকা (header) ছাড়াই।

১. আঁকার সাধারণ হাতিয়ার (shared drawing helpers)

এই ৫টা ফাংশন কোনো নির্দিষ্ট চরিত্র বা দৃশ্যের না — এগুলো হলো ছোট ছোট মৌলিক হাতিয়ার, যা দিয়ে বাকি সব কিছুর আঁকা হয়।

`drawRect(x1, y1, x2, y2)`

গোপন

ছবি আঁকা

দুটো কোণার বিন্দু ($x1, y1$) আর ($x2, y2$) দিয়ে বোঝানো একটা রঙিন চতুর্ভুজ (আয়তক্ষেত্র) আঁকে। এটা এই পুরো প্রোজেক্টের সবচেয়ে বেশি ব্যবহৃত হাতিয়ারগুলোর একটা — ইট, পাথর, শরীরের অংশ, প্রায় সবকিছুই এই চতুর্ভুজ দিয়ে তৈরি।

অন্য কোনো প্রজেক্ট-ফাংশন ডাকে না

`drawCircle(cx, cy, r)`

গোপন

ছবি আঁকা

কেন্দ্র (cx, cy) আর ব্যাসার্ধ r দিয়ে একটা ভরাট বৃত্ত আঁকে। আসলে সত্যিকারের গোল বৃত্ত না — ৪০টা ছোট ছোট বিন্দু দিয়ে একটা বহুভুজ বানিয়ে বৃত্তের মতো দেখানো হয়, কিন্তু চোখে সেটা মসৃণ গোল-ই লাগে।

অন্য কোনো প্রজেক্ট-ফাংশন ডাকে না

`drawText(x, y, text)`

গোপন

ছবি আঁকা

(x,y) জায়গা থেকে শুরু করে সাধারণ লেখা (যেমন "Score: 5") আঁকে, GLUT-এর তৈরি রেডিমেড অক্ষর ব্যবহার করে। Score, Lives আর "Press M for menu" — সবই এই ফাংশন দিয়ে লেখা হয়।

অন্য কোনো প্রজেক্ট-ফাংশন ডাকে না

`drawStrokeText(centerX, y, scale, text)`

public

ছবি আঁকা

এটাও লেখা আঁকে, কিন্তু ভিন্নভাবে — অক্ষরগুলো রেখা (স্ট্রোক/স্কেলযোগ্য) দিয়ে আঁকা হয়, তাই এগুলো বড় করা যায় এবং মেনুর "STONE RUNNER" শিরোনামের মতো বড়, সুন্দর করে দেখানো যায়। লেখাটার মোট প্রস্থ মেপে সেটাকে *centerX* বরাবর ঠিক মাঝ বরাবর বসায়।

অন্য কোনো প্রজেক্ট-ফাংশন ডাকে না

`drawTextCentered(centerX, y, text)`

public

ছবি আঁকা

সাধারণ বিটম্যাপ লেখার প্রস্থ মেপে নিয়ে সেটাকে ঠিক মাঝ বরাবর বসিয়ে আঁকে। মেনুর "Choose an animation" এবং অপশনের লেখাগুলো এভাবেই মাঝ বরাবর দেখানো হয়।

এটি যাদের ডাকে: → `drawText`

২. মেনু (menu)

`drawMenu()`

public

ছবি আঁকা

খেলা শুরু হলে যে প্রথম কালো-সোনালি প্যানেলটা দেখা যায়, সেটা এই ফাংশনই আঁকে — একটা গাঢ় পটভূমি, চারপাশে সোনালি ফ্রেম, বড় করে "STONE RUNNER" নাম, আর নিচে "1 - DreamWorld" ও "2 - DreamHell" অপশন দুটো।

এটি যাদের ডাকে: → `drawRect`, `drawStrokeText`, `drawTextCentered`

৩. খেলার বস্তু আঁকা (game objects)

drawCoin()

গোপন

ছবি আঁকা

একটা সোনালি কয়েন আঁকে — তিনটা ভিন্ন রঙের বৃত্ত একটার ওপর একটা বসিয়ে, তারপর একটা "+"-এর মতো ছোট রেখা আর একটা চকচকে হাইলাইট বিন্দু দিয়ে সাজিয়ে। কয়েনটা যেখানে বসানো হবে, সেই জায়গার হিসাব drawGame() আগে থেকেই ঠিক করে দেয়।

এটি যাদের ডাকে: → drawCircle

drawStone()

গোপন

ছবি আঁকা

রাস্তার বাধা — একটা ছাই-ধূসর পাথর আঁকে, একটা ভরাট বহুভুজ আকারে, চারপাশে একটু গাঢ় রেখার আউটলাইন দিয়ে। এই পাথরে ধাক্কা লাগলেই লাইফ কমে যায়।

অন্য কোনো প্রজেক্ট-ফাংশন ডাকে না

8. সংঘর্ষ পরীক্ষা (collision detection)

rectHit(a1,b1,a2,b2, c1,d1,c2,d2)

গোপন

হিসাব

দুইটা চতুর্ভুজ (বক্স) একে অপরকে ছুঁয়েছে কিনা, সেটা হিসাব করে বলে দেয় — true মানে "ছুঁয়েছে", false মানে "ছোঁয়নি"। এই একটা ফাংশনই কয়েন কুড়ানো আর পাথরে ধাক্কা লাগা, দুটোই পরীক্ষা করতে ব্যবহার হয়।

অন্য কোনো প্রজেক্ট-ফাংশন ডাকে না

checkCoins()

গোপন

সংঘর্ষ পরীক্ষা

মেয়েটার শরীরের বক্স আর প্রতিটা জীবিত কয়েনের বক্স মিলিয়ে দেখে — ছোঁয়া লাগলে সেই কয়েনটা "মৃত" (coinAlive = false) করে দেয় আর স্কোর ১ বাড়িয়ে দেয়।

এটি যাদের ডাকে: → rectHit, playerGetY (roshni_player মডিউল থেকে)

checkStones()

গোপন

সংঘর্ষ পরীক্ষা

মেয়েটা কোনো পাথরে ধাক্কা খেয়েছে কিনা দেখে। ধাক্কা লাগলে ১টা লাইফ কমায়, স্কোর থেকে ২ পয়েন্ট বাদ দেয় (০-এর নিচে যেতে দেয় না), আর ৪৫ ফ্রেমের জন্য একটা "নিরাপদ সময়" (hitTimer) চালু করে, যাতে পরপর কয়েকবার ধাক্কা না লাগে। লাইফ শূন্যে নেমে গেলে gameState-কে ৩ ("ধরা পড়েছে") বানিয়ে দেয়।

এটি যাদের ডাকে: → rectHit, playerGetY (roshni_player মডিউল থেকে)

৫. খেলার অবস্থা ও রিসেট (state & reset)

resetRun()

গোপন

অবস্থা/রিসেট

একটা নতুন দৌড় শুরু করার সব প্রস্তুতি নেয়: স্কোর-লাইফ-গতি আবার শুরুর মানে ফেরায়, ডাইনোসর আর মেয়েটাকে তাদের শুরুর জায়গায় পাঠায়, আর সব পাথর-কয়েনকে নতুন করে সাজায়। প্রথমবার ডাকা হলে এলোমেলো সংখ্যার জেনারেটরও (random seed) চালু করে দেয়, যাতে প্রতিবার খেলাটা একটু ভিন্ন লাগে।

এটি যাদের ডাকে: → dinoReset (dragon), playerReset (roshni_player)

৬. প্রতি ফ্রেমে আঁকা ও চালানো (per-frame draw & update)

drawGameUI()

গোপন

ছবি আঁকা

স্কোর আর লাইফের সংখ্যা লেখে, আর অবস্থা (gameState) অনুযায়ী আলাদা আলাদা বার্তা দেখায়: শুরুতে "Press ENTER to Start", ধরা পড়লে "CAUGHT!", আর গেম ওভার হলে "GAME OVER" ও "Press ENTER to Restart"।

এটি যাদের ডাকে: → drawText

drawGame(groundOffset)

গোপন

ছবি আঁকা

DreamWorld ও DreamHell — দুটো দৃশ্যই খেলার সাধারণ অংশটা (কয়েন, পাথর, ডাইনোসর, মেয়েটা, UI লেখা) এই একটাই ফাংশন আঁকে, শুধু *groundOffset* মান দিয়ে সামান্য উঁচু-নিচু সমন্বয় করে। যদি `gameState == 1` (খেলা চলছে) হয়, তাহলে ডাইনোসরকে পেছনে আর মেয়েটাকে সামনে রেখে আঁকে; নাহলে (ধরা পড়া/গেম ওভার অবস্থায়) মেয়েটাকে আগে আর ডাইনোসরকে তার ওপরে আঁকে "ধরে ফেলার" দৃশ্যটা বোঝায়।

এটি যাদের ডাকে: → `drawCoin`, `drawDino (dragon)`, `drawStone`, `drawPlayer (roshni_player)`, `playerGetY (roshni_player)`, `drawGameUI`

animateGame()

গোপন

নড়াচড়া

এই ফাংশনটাই আসল খেলার ইঞ্জিন। খেলা চলাকালীন (`gameState == 1`): দৌড়ের গতি স্কোরের সাথে বাড়ায়, পাথর-কয়েনদের বামে সরিয়ে দরকার হলে ডানে ফিরিয়ে আনে, কয়েনগুলোকে ওপর-নিচ দোলায়, মেয়েটা লাফাচ্ছে কিনা দেখে লাফের হিসাব আপডেট করে, ছজনের পা-নাড়ানো (*pose*) অ্যানিমেশন চালায়, নিরাপদ-সময় (*hitTimer*) কমায়, আর কয়েন/পাথরের সংঘর্ষ পরীক্ষা করে। ধরা পড়া অবস্থায় (`gameState == 3`) প্রতি ফ্রেমে ডাইনোসরকে এগিয়ে দেয়।

এটি যাদের ডাকে: → `playerIsJumping`, `playerJumpUpdate`, `playerAnimate (roshni_player)`, `dinoAnimate`, `dinoChase (dragon)`, `checkCoins`, `checkStones`

৭. ইনপুট (কীবোর্ড) (input handling)

gameKeyPress(key)

গোপন

ইনপুট

শুধু দৃশ্য চালু থাকলে (DreamWorld/DreamHell-এ থাকলে) কাজ করে। ENTER (কোড ১৩) চাপলে — যদি খেলাটা শুরুর আগে বা গেম ওভার অবস্থায় থাকে — নতুন করে দৌড় শুরু করে দেয়। SPACE চাপলে, খেলা সত্যিই চলছে (`gameState == 1`) এমন অবস্থায়, লাফানো শুরু করে।

এটি যাদের ডাকে: → `resetRun`, `playerJumpStart (roshni_player)`

৮. GLUT সংযোগ ও প্রবেশ পয়েন্ট (GLUT wiring & entry point)

display()

public – GLUT কলব্যাক

ছবি আঁকা

GLUT যখনই মনে করে পর্দা নতুন করে আঁকা দরকার, তখন এই ফাংশনকে ডাকে। selected কোন দৃশ্য বলছে সেটা দেখে হয় মেনু আঁকে, নয়তো দৃশ্য (DreamWorld/DreamHell) আর তার ওপরে খেলার অংশ আঁকে, তারপর "Press M for menu" লেখাটা বসিয়ে শেষে glutSwapBuffers() দিয়ে পর্দাটা দেখায়।

এটি যাদের ডাকে: → drawBackground (DreamWorld), drawDreamHell (DreamHell), drawGame, drawMenu, drawText

update(value)

public – GLUT কলব্যাক

নড়াচড়া

GLUT-এর ১৬ মিলিসেকেন্ডের ঘড়ি — এটাই আসলে গোটা খেলাটাকে "জীবন্ত" রাখে। কোন দৃশ্য চালু আছে সেটা দেখে DreamWorld বা DreamHell-এর অ্যানিমেশন এগিয়ে নেয়, তারপর animateGame() ডাকে, তারপর পর্দা আবার আঁকতে বলে (glutPostRedisplay) আর নিজেকে আবার ১৬ms পরে ডাকতে বলে (glutTimerFunc) — এভাবেই লুপটা চলতেই থাকে।

এটি যাদের ডাকে: → backgroundAnimate (DreamWorld), dreamHellAnimate (DreamHell), animateGame

keyboard(key, x, y)

public – GLUT কলব্যাক

ইনপুট

যেকোনো কী চাপলেই GLUT এটাকে ডাকে। '1' চাপলে DreamWorld দৃশ্য, '2' চাপলে DreamHell দৃশ্য বেছে নেয়; দৃশ্য চালু থাকলে বাকি কাজ gameKeyPress()-কে দিয়ে দেয়; 'm'/'M' চাপলে আবার মেনুতে ফেরত যায়; আর ESC (কোড ২৭) চাপলে পুরো প্রোগ্রাম বন্ধ করে দেয়।

এটি যাদের ডাকে: → gameKeyPress

init()

public

শুরু করা

প্রোগ্রাম চালু হওয়ার সময় একবারই কাজ করে — পর্দার পটভূমির রঙ (সাদা), স্বচ্ছতা/ব্লেন্ডিং, আর ক্যামেরার ফ্ল্যাট ২D দৃষ্টিকোণ (gluOrtho2D) ঠিক করে দেয়। শেষে resetRun() ডেকে প্রথম দৌড়ের জন্য সব প্রস্তুত করে রাখে।

এটি যাদের ডাকে: → resetRun

main(argc, argv)

প্রবেশ পয়েন্ট

শুরু করা

প্রোগ্রামের প্রথম ফাংশন। GLUT চালু করে, ৮০০x৪৫০ সাইজের জানালা বানায়, init() ডেকে সব প্রস্তুত করে, display/keyboard/update-কে GLUT-এর কলব্যাক হিসেবে নিবন্ধন করে, আর শেষে glutMainLoop() ডেকে প্রোগ্রামের নিয়ন্ত্রণ GLUT-এর হাতে ছেড়ে দেয়।

এটি যাদের ডাকে: → init

DreamWorld.cpp — দিনের দৃশ্যের সব ফাংশন

১৩টা ফাংশন। এই ফাইলটা শুধু ছবি আঁকে আর সেই ছবিকে স্ক্রল (scroll) করায় — এখানে কোনো স্কোর, লাইফ বা সংঘর্ষের হিসাব নেই।

যখন মেনু থেকে 1 চাপা হয়, তখন এই ফাইলটাই আকাশ, সূর্য, মেঘ, পাহাড়, গাছ, ফুল আর ঘাসের মাটি — সব মিলিয়ে সুন্দর দিনের দৃশ্যটা বানায়। এর ভেতরের বেশিরভাগ ফাংশনই "গোপন" (static) — শুধু **drawBackground()** আর **backgroundAnimate()** — এই দুটোই বাইরে (main.cpp থেকে) ডাকা যায়, কারণ শুধু এই দুটোর নামই DreamWorld.h ফাইলে লেখা আছে।

একটা চালাকি — "দুইবার আঁকো, একবার বামে একবার ডানে": দৃশ্যটা যখন বামে সরে যায় (স্ক্রল হয়), তখন একটা কপি স্ক্রিনের বাইরে চলে যাওয়ার সাথে সাথেই আরেকটা কপি ঠিক তার পাশে এসে হাজির হয় — তাই সবসময় স্ক্রিন ভরা থাকে, কোনো ফাঁকা জায়গা চোখে পড়ে না। এই জন্যই **drawBackground()** মেঘ, পাহাড়, ঝোপ আর গাছ — প্রতিটাকে দুইবার করে আঁকে, শুধু একটু আলাদা জায়গায়।

drawBackground() — সব কিছু একসাথে জোড়া লাগায়

drawBackground()



এই ক্রমে (পেছন থেকে সামনে) আঁকা হয় — কিছু ফাংশন স্ক্রলের জন্য দুইবার ডাকা হয়:

১. drawSky

২. drawSun

৩. drawClouds

৪. drawMountains

৫. drawBackgroundBushes

৬. drawTree

৭. drawBushes

৮. drawFlowers

৯. drawGround

drawBackground()

public

ছবি আঁকা

প্রতি ফ্রেমে পুরো DreamWorld দৃশ্যটা আঁকার একমাত্র দরজা — main.cpp শুধু এই একটা ফাংশনকেই ডাকে। ওপরের তালিকার প্রতিটা ফাংশনকে ক্রমানুসারে ডেকে, কয়েকটাকে দুইবার (একটু ভিন্ন জায়গায় সরিয়ে) ডেকে, পুরো দৃশ্যটা একসাথে সাজিয়ে দেয়।

এটি যাদের ডাকে: → drawSky, drawSun, drawClouds, drawMountains, drawBackgroundBushes, drawTree, drawBushes, drawFlowers, drawGround

backgroundAnimate(speed, running)

public

নড়াচড়া

দৃশ্যটাকে সচল রাখার ইঞ্জিন — ৬টা "স্কল-দূরত্ব" (কতটুকু সরে গেছে) বাড়িয়ে দেয়। মেঘ সবসময় সরে (এমনকি খেলা থেমে থাকলেও), কিন্তু পাহাড়/ঝোপ/গাছ/ইট/ঘাস শুধু running সত্যি হলে সরে, আর প্রতিটা আলাদা গতিতে সরে (কাছেরটা দ্রুত, দূরেরটা ধীরে) — এই জন্যই দৃশ্যটাকে গভীর/থ্রিডি মনে হয় (একে বলে parallax)।

অন্য কোনো প্রজেক্ট-ফাংশন ডাকে না — শুধু ৬টা সংখ্যা বাড়ায়/মোড়ায়

আকাশ ও সূর্য

drawSky()

গোপন

ছবি আঁকা

পুরো স্ক্রিন জুড়ে গাঢ় নীল থেকে হালকা আকাশি রঙে মিলিয়ে যাওয়া একটা আকাশ আঁকে — সবচেয়ে পেছনের স্তর, বাকি সব কিছুর আগে আঁকা হয়।

অন্য কোনো প্রজেক্ট-ফাংশন ডাকে না

drawSun()

গোপন

ছবি আঁকা

আকাশের একটা নির্দিষ্ট জায়গায় স্থির সূর্য আঁকে — তিনটা ভিন্ন রঙের বৃত্ত দিয়ে জ্বলজ্বলে ভাব বানিয়ে, চারপাশে ৮টা ছোট রশ্মি রেখা দিয়ে।

এটি যাদের ডাকে: → drawCircle

মেঘ ও পাহাড়

drawClouds()

গোপন

ছবি আঁকা

৪টা মেঘের গুচ্ছ আঁকে, প্রতিটা কয়েকটা বৃত্ত দিয়ে তৈরি — আগে একটু গাঢ় ছায়া, তার ওপর সাদা হাইলাইট, যাতে মেঘগুলোকে ফুলে থাকা তুলোর মতো লাগে।

এটি যাদের ডাকে: → drawCircle, drawRect

drawMountains()

গোপন

ছবি আঁকা

৪টা ত্রিভুজ আকৃতির পাহাড় আঁকে, প্রতিটার একপাশ হালকা রঙের আর একপাশ গাঢ় (ছায়ার দিক দেখাতে), আর চূড়ায় সাদা বরফের ছোঁয়া।

অন্য কোনো প্রজেক্ট-ফাংশন ডাকে না

গাছপালা

drawBackgroundBushes()

গোপন

ছবি আঁকা

দূরের ৫টা ছোট নিচু ঝোপ আঁকে (মূল গাছের পেছনে থাকা স্তর), গাঢ় সবুজ বৃত্তের ওপর হালকা সবুজ হাইলাইট দিয়ে।

এটি যাদের ডাকে: → drawCircle

drawTree()

গোপন

ছবি আঁকা

একটা বড় গাছ আঁকে — বাদামি গুঁড়ি, ৪টা মোটা ডাল, আর কয়েক স্তরের সবুজ পাতা (তিনটা ভিন্ন সবুজ শেড দিয়ে গভীরতা বোঝানো)।

এটি যাদের ডাকে: → drawCircle

drawBushes()

গোপন

ছবি আঁকা

সামনের দিকে ৩টা ঝোপের গুচ্ছ আঁকে, গাছের সাথেই একই দলে থেকে স্ক্রল হয়।

এটি যাদের ডাকে: → drawCircle

drawFlowers()

গোপন

ছবি আঁকা

৩টা ফুলের গুচ্ছ আঁকে, প্রতিটা ৪টা গোলাপি পাপড়ি আর মাঝে একটা হলুদ কেন্দ্র দিয়ে।

এটি যাদের ডাকে: → drawCircle

মাটি ও রাস্তা

drawBrick()

গোপন

ছবি আঁকা

একটামাত্র ইট আঁকে, নিজের জায়গার (0,0) কেন্দ্র বরাবর — ৪টা স্তরের রঙ দিয়ে যাতে হালকা থ্রিডি ভাব আসে। drawGround() এটাকে বার বার ডেকে সারি সারি ইট বানায়।

এটি যাদের ডাকে: → drawRect

drawGrassBlades()

গোপন

ছবি আঁকা

দুইটা ছোট ত্রিভুজ ঘাসের পাতা আঁকে একসাথে, একটা গুচ্ছ হিসেবে। drawGround() এটাকেও বার বার ডেকে সবুজ মাটির ওপর সারি সারি ঘাস বানায়।

অন্য কোনো প্রজেক্ট-ফাংশন ডাকে না

drawGround()

গোপন

ছবি আঁকা

সবচেয়ে নিচের মাটির স্তর আঁকে — প্রথমে একটা বাদামি ভিত্তি, তার ওপর ৩ সারি ইট (ইটের প্যাটার্ন সারিতে সারিতে একটু সরানো, যাতে সত্যিকারের ইটের দেয়ালের মতো লাগে), আর সবার ওপরে সবুজ ঘাসের সারি। মেয়েটা আর ডাইনোসর এই মাটির ওপর দিয়েই দৌড়ায়।

এটি যাদের ডাকে: → drawRect, drawBrick, drawGrassBlades

DreamHell.cpp – মরুভূমির দৃশ্যের সব ফাংশন

১৮টা ফাংশন। নাম শুনে ভয় লাগলেও, "Hell" এখানে আসলে গরম মরুভূমি বোঝাতে ব্যবহার হয়েছে — আগুন বা রাক্ষস কিছু নেই!

যখন মেনু থেকে 2 চাপা হয়, তখন এই ফাইলটা বালির রঙের আকাশ, ঘুরন্ত সূর্য, মেঘ, উড়ন্ত পাখি, দূরের পাহাড়, শুকনো মাটি, ক্যাকটাস, একটা গরুর মাথার খুলি, আর গড়িয়ে চলা কাঁটারোপ (tumbleweed) দিয়ে দৃশ্যটা বানায়। শুধু **drawDreamHell()** আর **dreamHellAnimate()** — এই দুটোই বাইরে থেকে ডাকা যায়।

DreamWorld থেকে ভিন্ন একটা কৌশল: **DreamWorld.cpp** ৬টা আলাদা "কতদূর সরেছে" সংখ্যা রাখে। **DreamHell.cpp** তার বদলে একটামাত্র ঘড়ি (**gTime**) রাখে, আর প্রতিটা জিনিসের অবস্থান হিসাব করে নেয় **wrapX()** নামের একটা গণিত-ফাংশন দিয়ে — এটাই সময় আর গতি থেকে বলে দেয় জিনিসটা এখন স্ক্রিনের কোথায় থাকবে।

drawDreamHell() – সব কিছু একসাথে জোড়া লাগায়

drawDreamHell()



এই ক্রমে (পেছন থেকে সামনে) আঁকা হয়:

১. drawSky

২. drawSun

৩. drawCloudRow

৪. drawBirdRow

৫. drawMountains

৬. drawSmallCactus

৭. drawGround

৮. drawRock

৯. drawCactus

১০. drawSkull

১১. drawTumbleweed

১২. wrapX

drawDreamHell()

public

ছবি আঁকা

প্রতি ফ্রেমে পুরো মরুভূমির দৃশ্যটা আঁকার একমাত্র দরজা — আকাশ থেকে শুরু করে সবচেয়ে সামনের কাঁটাঝোপ পর্যন্ত, স্তরে স্তরে। প্রতিটা স্ক্রল হওয়া জিনিসের জায়গা wrapX() দিয়ে হিসাব করা হয়, যাতে বিভিন্ন স্তর বিভিন্ন গতিতে সরে (গভীরতার অনুভূতি তৈরি করতে)।

এটি যাদের ডাকে: → drawSky, drawSun, drawCloudRow, drawBirdRow, drawMountains, drawSmallCactus, drawGround, drawRock, drawCactus, drawSkull, drawTumbleweed, wrapX

dreamHellAnimate()

public

নড়াচড়া

দৃশ্যের একমাত্র ঘড়ি **gTime**-কে প্রতি টিকে একটু বাড়ায় (০.০১৬ করে, প্রায় ৬০ ফ্রেম/সেকেন্ডের হিসাবে), আর সূর্যের ঘোরার কোণ **sunAngle**-কেও একটু বাড়ায়। এই দুটো সংখ্যা বদলানোর কারণেই wrapX()-এর হিসাব করা প্রতিটা জিনিসের জায়গা আর সূর্যের ঘোরা — দুটোই সময়ের সাথে বদলে যায়।

অন্য কোনো প্রজেক্ট-ফাংশন ডাকে না

স্ক্রলের হিসাব ও মৌলিক আকৃতি

wrapX(x0, speed, margin)

গোপন

হিসাব

সময় (gTime) আর গতি থেকে হিসাব করে বলে দেয় একটা জিনিস এখন স্ক্রিনের ঠিক কোন x-জায়গায় থাকবে, আর জিনিসটা বাম দিকে বেরিয়ে গেলে আবার ডান দিক থেকে ফিরিয়ে আনে (wrap করে) — যাতে চলাটা কখনো থামতে বা লাফাতে না দেখায়। এই ফাংশনটাই DreamHell.cpp-এর প্রায় সব স্ক্রলিং জিনিসের ভিত্তি।

অন্য কোনো প্রজেক্ট-ফাংশন ডাকে না

drawEllipse(cx, cy, rx, ry)

গোপন

ছবি আঁকা

একটা ডিম্বাকৃতি (ellipse) ভরাট আকৃতি আঁকে, ২৮টা বিন্দু দিয়ে বহুভুজ বানিয়ে। এটা মেঘ, সূর্যের শরীর, আর মাটির নুড়ি-পাথর আঁকতে বার বার ব্যবহার হয়।

অন্য কোনো প্রজেক্ট-ফাংশন ডাকে না

আকাশ ও সূর্য

drawSky()

গোপন

ছবি আঁকা

সবুজাভ-নীল থেকে হালকা আকাশি রঙে মিলিয়ে যাওয়া একটা স্থির আকাশ আঁকে, সবচেয়ে পেছনের স্তর হিসেবে।

অন্য কোনো প্রজেক্ট-ফাংশন ডাকে না

drawSun(cx, cy)

গোপন

ছবি আঁকা

১২টা ত্রিভুজ রশ্মি দিয়ে ঘেরা একটা সূর্য আঁকে, আর sunAngle মান অনুযায়ী পুরো সূর্যটাকে ঘুরিয়ে দেয় — তাই এই সূর্যটা DreamWorld-এর সূর্যের মতো স্থির নয়, বরং আস্তে আস্তে নিজের জায়গায় ঘোরে।

এটি যাদের ডাকে: → drawEllipse

মেঘ ও পাখি

drawCloud(x, y, size)

গোপন

ছবি আঁকা

একটা মেঘ আঁকে, তিনটা ওভারল্যাপ করা ডিম্বাকৃতি দিয়ে তৈরি। এটা নিজে নড়ে না — তার অবস্থান কোথায় বসবে, সেটা drawCloudRow() ঠিক করে দেয়।

এটি যাদের ডাকে: → drawEllipse

drawCloudRow(y, size, gap, speed)

গোপন

নড়াচড়া

একটা পুরো সারি মেঘ আঁকে আর নিজে থেকেই স্ক্রল করায় — কতগুলো মেঘ লাগবে হিসাব করে, প্রতিটার জায়গা wrapX-এর মতো নিজস্ব হিসাবে ঠিক করে, ফাঁকা জায়গা ছাড়াই সারিটা চক্রাকারে চলতে থাকে। drawDreamHell() এটাকে ৩ বার ডাকে (কাছের/মাঝের/দূরের সারি বানাতে)।

এটি যাদের ডাকে: → drawCloud

drawBird(x, y, size, wing)

গোপন

ছবি আঁকা

একটা পাখি আঁকে, দুটো রেখা দিয়ে "V" আকৃতির ডানা হিসেবে। *wing* মান বদলালে ডানার কোণ বদলে যায়, তাই এটাই ডানা ঝাপটানোর ভঙ্গি তৈরি করে।

অন্য কোনো প্রজেক্ট-ফাংশন ডাকে না

drawBirdRow(y, size, gap, speed)

গোপন

নড়াচড়া

একঝাঁক পাখি আঁকে আর স্ক্রল করায় (drawCloudRow-এর মতোই কৌশলে), সাথে প্রতিটা পাখির ডানার কোণ sin() (একটা ডেউয়ের মতো ওঠা-নামা করা গণিত ফাংশন) দিয়ে হিসাব করে, যাতে সব পাখি একসাথে না ঝাপটে একটু আলাদা আলাদা সময়ে ঝাপটায়।

এটি যাদের ডাকে: → drawBird

পাহাড়

drawMountain(x, bottom, width, height, r, g, b)

গোপন

ছবি আঁকা

একটা ত্রিভুজ আকৃতির পাহাড় আঁকে, দেওয়া রঙ দিয়ে। এটা একটা সাধারণ হাতিয়ার — drawMountains() একে বার বার ডেকে পুরো পাহাড়ের সারি বানায়।

অন্য কোনো প্রজেক্ট-ফাংশন ডাকে না

drawMountains()

গোপন

ছবি আঁকা

১০টা পাহাড়চূড়া আঁকে — ৫টা দূরের হালকা রঙের সারি, ৫টা কাছের গাঢ় রঙের সারি — যাতে গভীরতার একটা অনুভূতি হয়। এই পাহাড়গুলো স্থির থাকে, স্ক্রল হয় না।

এটি যাদের ডাকে: → drawMountain

মাটি ও মরুভূমির জিনিসপত্র

drawGround()

গোপন

ছবি আঁকা

মাটির স্তর আঁকে — নিচে গাঢ় বাদামি মাটি (তার ওপর wrapX দিয়ে স্ক্রল হওয়া ১৪টা ছোট নুড়ি-পাথর ছড়ানো), আর তার ওপরে হালকা বালির স্তর, যার ওপর ক্যাকটাস আর পাথর দাঁড়িয়ে থাকে।

এটি যাদের ডাকে: → drawRect, wrapX, drawEllipse

drawRock(x, y, w, h)

গোপন

ছবি আঁকা

একটা কোণায়ুক্ত পাথর আঁকে, ৬-কোণা বহুভুজ আকারে, বাদামি রঙে।

অন্য কোনো প্রজেক্ট-ফাংশন ডাকে না

drawCactus(x, bottom)

গোপন

ছবি আঁকা

একটা বড় ক্যাকটাস আঁকে — একটা লম্বা মূল কাণ্ড, আর দুই পাশে একটা করে বাছ, সবুজ রঙে।

অন্য কোনো প্রজেক্ট-ফাংশন ডাকে না

drawSmallCactus(x, bottom)

গোপন

ছবি আঁকা

একটা ছোট ক্যাকটাস আঁকে, ছোট কাণ্ড আর ছোটো ছোট বাছ দিয়ে — দূরের/ব্যাকগ্রাউন্ডের অনেকগুলো ক্যাকটাসের জন্য ব্যবহার হয়।

অন্য কোনো প্রজেক্ট-ফাংশন ডাকে না

drawTumbleweed(cx, groundY, r, speed)

গোপন

নড়াচড়া

একটা কাঁটাঝোপের গোলা আঁকে যেটা বাউন্স করে (sin() দিয়ে লাফানোর উচ্চতা হিসাব করে) আর ঘোরে (স্পিন করে), সাথে নিচে একটা ছায়া আঁকে যেটা লাফানোর সময় ছোট হয়ে যায় — খুব বাস্তব একটা গড়িয়ে চলার অনুভূতি তৈরি করে।

অন্য কোনো প্রজেক্ট-ফাংশন ডাকে না

drawSkull(x, y)

গোপন

ছবি আঁকা

একটা গরুর মাথার খুলি আঁকে — মাথা, ঝুলন্ত নাক-অংশ, দুটো শিং, আর দুটো গাঢ় চোখের গর্ত — সাদা-হাড় রঙে, মরুভূমির একটা সাজসজ্জার অংশ হিসেবে।

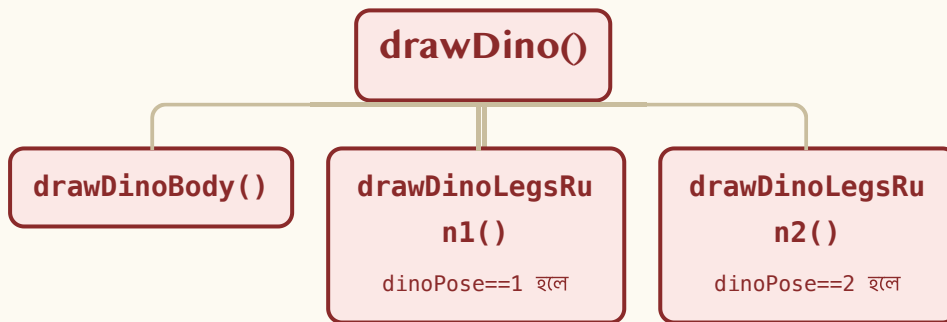
অন্য কোনো প্রজেক্ট-ফাংশন ডাকে না

dragon.cpp – ডাইনোসরের সব ফাংশন

৮টা ফাংশন। এই ফাইলটা ত্যাঁড়া করা ডাইনোসর চরিত্রটার আঁকা, দৌড়ের অ্যানিমেশন, আর ত্যাঁড়া করার নড়াচড়া সামলায়।

ডাইনোসরের ৩টা "গোপন" (static) সংখ্যা আছে যেগুলো এই ফাইলের বাইরে থেকে সরাসরি দেখা যায় না: **dinoX** (এখন কোথায় আছে), **dinoPose** (কোন পায়ের ভঙ্গি দেখানো হচ্ছে — ১ বা ২), আর **dinoCounter** (পরের ভঙ্গি বদলাতে আর কত ফ্রেম বাকি)। বাইরের জগৎ (main.cpp) এগুলো সরাসরি না ছুঁয়ে, শুধু dragon.h-এ লেখা ৫টা পাবলিক ফাংশন দিয়েই ডাইনোসরের সাথে কথা বলে।

drawDino() – শরীর ও পা জোড়া লাগায়



drawDino()

public

ছবি আঁকা

ডাইনোসরের পুরো ছবিটা আঁকার একমাত্র দরজা। সবসময় আগে শরীর আঁকে, তারপর **dinoPose** দেখে ঠিক করে কোন পায়ের ভঙ্গিটা এখন দেখাবে — তাই সময়ের সাথে সাথে পা দুটো বদলে বদলে "দৌড়ানোর" মতো দেখায়।

এটি যাদের ডাকে: → drawDinoBody, drawDinoLegsRun1, drawDinoLegsRun2

drawDinoBody()

গোপন

ছবি আঁকা

ডাইনোসরের শরীর, ঘাড়, মাথা, লেজ — সব কয়েকটা চতুর্ভুজ দিয়ে বাদামি-কমলা রঙে আঁকে, আর শেষে একটা ছোট কালো বিন্দু দিয়ে চোখ/নাক ফুটিয়ে তোলে।

এটি যাদের ডাকে: → drawRect

drawDinoLegsRun1()

গোপন

ছবি আঁকা

দৌড়ানোর প্রথম পায়ের ভঙ্গি আঁকে — ৪টা চতুর্ভুজ দিয়ে দুই পায়ের উপর-নিচের অংশ।

এটি যাদের ডাকে: → drawRect

drawDinoLegsRun2()

গোপন

ছবি আঁকা

দৌড়ানোর দ্বিতীয় পায়ের ভঙ্গি আঁকে, প্রথম ভঙ্গির চেয়ে একটু ভিন্ন জায়গায় — এই দুটো ভঙ্গি পালাক্রমে দেখালেই পা নড়ছে মনে হয়।

এটি যাদের ডাকে: → drawRect

অবস্থান ও নড়াচড়া

dinoGetX()

public

হিসাব

ডাইনোসর এখন x-অক্ষে কোথায় আছে, সেই সংখ্যাটা (dinoX) ফেরত দেয়। main.cpp এটা ব্যবহার করে জানে কোথায় ডাইনোসরটা এঁকে দিতে হবে।

অন্য কোনো প্রজেক্ট-ফাংশন ডাকে না

dinoAnimate(speed)

public

নড়াচড়া

প্রতি ফ্রেমে dinoCounter বাড়ায়; একটা নির্দিষ্ট মান (গতি যত বেশি, ততই কম সময়) পার হলে dinoPose-কে ১ থেকে ২, বা ২ থেকে ১-এ পালেট দেয় — এটাই drawDino()-কে পা বদলাতে বলে।

অন্য কোনো প্রজেক্ট-ফাংশন ডাকে না

dinoChase()

public

নড়াচড়া

প্রতিবার ডাকা হলে dinoX-কে ৪ বাড়িয়ে দেয়, মানে ডাইনোসরটা একটু একটু করে সামনে এগোয়। dinoX ২০০ বা তার বেশি হয়ে গেলে true ফেরত দেয় — এটাই main.cpp-কে বলে দেয় "ধরে ফেলেছি, এবার Game Over দেখাও।"

অন্য কোনো প্রজেক্ট-ফাংশন ডাকে না

রিসেট

dinoReset()

public

অবস্থা/রিসেট

dinoX, dinoPose, dinoCounter — তিনটাকেই আবার শুরুর মানে ফিরিয়ে দেয় (dinoX=115, pose=1, counter=0)। নতুন দৌড় শুরু হওয়ার সময় main.cpp-এর resetRun() এটাকে ডাকে।

অন্য কোনো প্রজেক্ট-ফাংশন ডাকে না

roshni_player.cpp — মেয়েটির (রোশনী) সব ফাংশন

১০টা ফাংশন। এই ফাইলটা খেলোয়াড় চরিত্র রোশনীর আঁকা, দৌড়ের অ্যানিমেশন, আর সবচেয়ে গুরুত্বপূর্ণ — লাফের পদার্থবিজ্ঞান (jump physics) সামলায়।

রোশনীর ৫টা গোপন সংখ্যা আছে: **playerY** (তার পায়ের উচ্চতা — মাটিতে থাকলে ৯২), **jumpSpeed** (এই মুহুর্তে ওপরে/নিচে কত জোরে যাচ্ছে), **jumping** (লাফাচ্ছে কিনা), আর **playerPose/playerCounter** (দৌড়ের পায়ের ভঙ্গি বদলানোর জন্য, ডাইনোসরের মতোই)।

drawPlayer() — পা ও শরীর জোড়া লাগায়



drawPlayer()

public

ছবি আঁকা

রোশনীর পুরো ছবি আঁকার একমাত্র দরজা। **playerPose** দেখে পায়ের একটা ভঙ্গি বেছে আঁকে, তারপর সবসময় শরীর/জামা আঁকে তার ওপরে — যাতে জামাটা পায়ের ওপরের অংশ ঢেকে দেয় আর একটা মসৃণ, একরঙা চরিত্রের মতো লাগে।

এটি যাদের ডাকে: → drawPlayerLegs1, drawPlayerLegs2, drawPlayerBody

drawPlayerLegs1()

গোপন

ছবি আঁকা

দৌড়ের প্রথম পায়ের ভঙ্গি আঁকে — একটা পা সামনে, একটা পেছনে, দুই পায়ের নিচে জুতা।

অন্য কোনো প্রজেক্ট-ফাংশন ডাকে না

drawPlayerLegs2()

গোপন

ছবি আঁকা

দৌড়ের দ্বিতীয় পায়ের ভঙ্গি আঁকে — সামনের পা এবার একটু তোলা, পেছনের পা মাটিতে সোজা — যাতে ভঙ্গি ১-এর সাথে পালাক্রমে দেখালে দৌড়ানোর ভাব আসে।

অন্য কোনো প্রজেক্ট-ফাংশন ডাকে না

drawPlayerBody()

গোপন

ছবি আঁকা

পায়ের ওপরের সবকিছু আঁকে — গোলাপি জামা, মাথা, চুল, বাহু, চোখ-নাক-মুখ — এক গাদা রঙিন আকৃতি দিয়ে সাজিয়ে।

এটি যাদের ডাকে: → drawRect, drawCircle

লাফের পদার্থবিজ্ঞান (jump physics)

একটা লাফ কীভাবে কাজ করে: `playerJumpStart()` লাফটা শুরু করে (হঠাৎ ওপরের দিকে গতি দিয়ে)। এরপর প্রতি ফ্রেমে `playerJumpUpdate()` ডাকা হয়, যেটা মাধ্যাকর্ষণের (`gravity`) মতো করে ওপরের গতিটা ধীরে ধীরে কমিয়ে দেয়, যতক্ষণ না রোশনী আবার মাটিতে নেমে আসে। এই জন্যই লাফটা দেখতে একটা সুন্দর বাঁকা পথের (`arc`) মতো লাগে — সোজা ওপরে গিয়ে সোজা নিচে না নেমে।

playerJumpStart()

public

নড়াচড়া

লাফ শুরু করে — `jumping`-কে `true` বানায় আর `jumpSpeed`-কে ১০ বসিয়ে দেয় (হঠাৎ জোরে ওপরের দিকে গতি)। যদি রোশনী আগে থেকেই লাফাচ্ছে থাকে, তাহলে কিছুই করে না — তাই বাতাসে দ্বিতীয়বার লাফ দেওয়া যায় না।

অন্য কোনো প্রজেক্ট-ফাংশন ডাকে না

playerJumpUpdate()

public

নড়াচড়া

প্রতি ফ্রেমে playerY-তে jumpSpeed যোগ করে (ওপরে/নিচে সরায়), তারপর jumpSpeed একটু কমিয়ে দেয় (মাধ্যাকর্ষণের নকল)। playerY আবার ৯২-এ বা তার নিচে নেমে এলে, রোশনীকে ঠিক মাটিতে বসিয়ে দেয় আর jumping বন্ধ করে দেয়।

অন্য কোনো প্রজেক্ট-ফাংশন ডাকে না

playerGetY()

public

হিসাব

রোশনীর এই মুহূর্তের উচ্চতা (playerY) ফেরত দেয়। main.cpp এটা ব্যবহার করে জানে তাকে কোথায় আঁকতে হবে, আর সংঘর্ষ পরীক্ষার (checkCoins/checkStones) হিসাবেও এটা লাগে।

অন্য কোনো প্রজেক্ট-ফাংশন ডাকে না

playerIsJumping()

public

হিসাব

রোশনী এখন বাতাসে আছে কিনা (jumping) সেটা true/false দিয়ে ফেরত দেয়। main.cpp-এর animateGame() এটা দেখে ঠিক করে playerJumpUpdate() ডাকা দরকার কিনা।

অন্য কোনো প্রজেক্ট-ফাংশন ডাকে না

দৌড়ের অ্যানিমেশন ও রিসেট

playerAnimate(speed)

public

নড়াচড়া

মাটিতে থাকলে, ডাইনোসরের মতোই playerCounter গুনে গুনে নির্দিষ্ট সময় পরপর playerPose বদলে দেয় (পা নড়াচড়ার ভান করতে)। কিন্তু বাতাসে থাকলে playerPose-কে ১-এ আটকে রাখে, যাতে লাফের সময় পা এলোমেলোভাবে না নড়ে।

অন্য কোনো প্রজেক্ট-ফাংশন ডাকে না

playerReset()

public

অবস্থা/রিসেট

রোশনীর সব অবস্থা শুরুর মানে ফিরিয়ে দেয় — মাটিতে দাঁড়ানো, লাফ বন্ধ, প্রথম পায়ের ভঙ্গি।
নতুন দৌড় শুরু হওয়ার সময় main.cpp-এর resetRun() এটাকে ডাকে।

অন্য কোনো প্রজেক্ট-ফাংশন ডাকে না

বাড়তি ৩টি ফাইল — একা একা চলা প্র্যাকটিস ফাইল

এই ৩টা ফাইল আসল খেলার (main.cpp + ৪টা মডিউল) সাথে যুক্ত না। প্রতিটার নিজের আলাদা main() আছে, তাই প্রতিটাই সম্পূর্ণ একা একা একটা প্রোগ্রাম হিসেবে চলতে পারে। যেহেতু এদের বেশিরভাগ ফাংশনের কাজ আগের অধ্যায়গুলোতে ব্যাখ্যা করা ফাংশনগুলোর মতোই (শুধু এক ফাইলে সব একসাথে গাদা করা), তাই এখানে প্রতিটা ফাংশনের নাম আর কাজ সংক্ষেপে একটা তালিকায় দেখানো হলো, বার বার একই ব্যাখ্যা না লিখে।

allInOne.cpp – ৭০টা ফাংশন

এই ফাইলটা আসলে পুরো খেলাটারই (main.cpp + DreamWorld + DreamHell + dragon + roshni_player) ছবছ একটা কপি, শুধু সব একটা ফাইলে জোড়া লাগানো, আর হেডার ফাইল কিছুই ব্যবহার হয়নি। জানালার নামও একই: "Vag Milka Vag"। নামের সংঘর্ষ এড়াতে DreamHell অংশের কিছু ফাংশনের নামের আগে "Hell" জোড়া হয়েছে (যেমন **drawHellSky**, **drawHellMountains**, **drawHellGround**), আর **drawSun** দুইবার লেখা হয়েছে ভিন্ন ভিন্ন প্যারামিটার দিয়ে (C++ একে "ওভারলোডিং" বলে অনুমতি দেয়)। বাকি সব ফাংশনের নাম আর কাজ ঠিক আগের অধ্যায়গুলোর (৬-১০) মতোই।

দল	ফাংশনগুলো (আগে যেখানে বিস্তারিত আছে)
আঁকার হাতিয়ার	drawRect, drawCircle, drawText — অধ্যায় ৬-এর মতোই
DreamWorld অংশ (দিন)	drawSky, drawSun, drawClouds, drawMountains, drawBackgroundBushes, drawTree, drawBushes, drawFlowers, drawBrick, drawGrassBlades, drawGround, drawBackground, backgroundAnimate — অধ্যায় ৭-এর মতোই
DreamHell অংশ (মরুভূমি)	wrapX, drawEllipse, drawHellSky, drawSun(cx,cy), drawCloud, drawCloudRow, drawBird, drawBirdRow, drawMountain, drawHellMountains, drawHellGround, drawRock, drawCactus, drawSmallCactus, drawTumbleweed, drawSkull, drawDreamHell, dreamHellAnimate — অধ্যায় ৮-এর মতোই (শুধু নামের আগে "Hell" জোড়া কিছু জায়গায়)
ডাইনোসর	drawDinoBody, drawDinoLegsRun1, drawDinoLegsRun2, drawDino, dinoGetX, dinoAnimate, dinoChase, dinoReset —

	অধ্যায় ৯-এর মতোই
রোশনী (খেলোয়াড়)	drawPlayerLegs1, drawPlayerLegs2, drawPlayerBody, drawPlayer, playerGetY, playerIsJumping, playerJumpStart, playerJumpUpdate, playerAnimate, playerReset — অধ্যায় ১০-এর মতোই
গেম কন্ট্রোলার ও GLUT	rectHit, drawCoin, drawStone, checkCoins, checkStones, drawGameUI, resetRun, drawGame, animateGame, gameKeyPress, drawStrokeText, drawTextCentered, drawMenu, display, update, keyboard, init, main — অধ্যায় ৬-এর মতোই

roshni.cpp – ৩৪টা ফাংশন

এটা মনে হয় খেলাটার একটা আগের খসড়া (draft) সংস্করণ — চরিত্র আর দৃশ্য তৈরির প্রথম দিককার প্র্যাকটিস, যেটা পরে আলাদা আলাদা ফাইলে (DreamWorld.cpp, dragon.cpp, roshni_player.cpp) ভাগ করে গোছানো হয়েছে। এখানে resetRun()-এর বদলে নাম **resetGame()**, আর drawGameUI()-এর বদলে নাম **drawUI()** — কিন্তু দুটোই একই কাজ করে।

দল	ফাংশনগুলো
আঁকার হাতিয়ার	drawRect, drawCircle, drawText — অধ্যায় ৬-এর drawRect/drawCircle/drawText-এর মতোই
দিনের দৃশ্য	drawSky, drawSun, drawClouds, drawMountains, drawBackgroundBushes, drawTree, drawBushes, drawFlowers, drawBrick, drawGrassBlades, drawGround — অধ্যায় ৭-এর DreamWorld ফাংশনগুলোর মতোই
খেলার বস্তু	drawCoin, drawStone — অধ্যায় ৬-এর drawCoin/drawStone-এর মতোই
রোশনী (খেলোয়াড়)	drawPlayerLegs1, drawPlayerLegs2, drawPlayerBody, drawPlayer — অধ্যায় ১০-এর draw ফাংশনগুলোর মতোই (তবে এই খসড়ায় আলাদা লাক-ফাংশন নেই)
ডাইনোসর	drawDinoBody, drawDinoLegsRun1, drawDinoLegsRun2, drawDino — অধ্যায় ৯-এর draw ফাংশনগুলোর মতোই
সংঘর্ষ ও অবস্থা	rectHit (অধ্যায় ৬-এর মতো), resetGame (resetRun()-এর মতোই কাজ করে), checkCoins , checkStones , drawUI (drawGameUI()-এর মতোই কাজ

	করে)
GLUT ও প্রবেশ পয়েন্ট	display, update, keyboard, init, main — অধ্যায় ৬-এর মতোই কাজ করে

referance.cpp – ২৪টা ফাংশন

এটাই সবচেয়ে সহজ, সবচেয়ে প্রথম দিককার খসড়া বলে মনে হয় — জানালায় শিরোনাম পর্যন্ত "Stone Runner - 2D Platformer" লেখা আছে। এখানে পুরো দৃশ্য (আকাশ-পাহাড়-গাছ) আঁকার বদলে শুধু একটা সাধারণ ধূসর রেখা (**drawGroundLine**) দিয়ে মাটি বোঝানো হয়েছে — বাকি সব মনোযোগ চরিত্র দুটো আর খেলার নিয়মের ওপর।

দল	ফাংশনগুলো
আঁকার হাতিয়ার	drawRect, drawCircle, drawText — অধ্যায় ৬-এর মতোই
মাটি	drawGroundLine — শুধু $y=৯২$ বরাবর একটা ধূসর সরল রেখা টানে, পুরো পর্দা জুড়ে, মাটি বোঝাতে
খেলার বস্তু	drawCoin, drawStone — অধ্যায় ৬-এর মতোই
রোশনী (খেলোয়াড়)	drawPlayerLegs1, drawPlayerLegs2, drawPlayerBody, drawPlayer — অধ্যায় ১০-এর draw ফাংশনগুলোর মতোই
ডাইনোসর	drawDinoBody, drawDinoLegsRun1, drawDinoLegsRun2, drawDino — অধ্যায় ৯-এর draw ফাংশনগুলোর মতোই
সংঘর্ষ ও অবস্থা	rectHit , resetGame (resetRun()-এর মতোই কাজ করে), checkCoins , checkStones , drawUI (drawGameUI()-এর মতোই কাজ করে)
GLUT ও প্রবেশ পয়েন্ট	display, update, keyboard, init, main — অধ্যায় ৬-এর মতোই কাজ করে

শেষ কথা

এই পুরো বইটা ঘুরে এসে, এখন তুমি জানো যে "স্টোন রানার" খেলাটা আসলে অনেকগুলো ছোট ছোট ফাংশনের একটা দল, যারা একসাথে মিলেমিশে কাজ করে। কোনো একটা ফাংশনই একা পুরো খেলাটা চালায় না — প্রতিটা ফাংশনের একটা করে ছোট নির্দিষ্ট কাজ আছে (কেউ শুধু বৃত্ত আঁকে, কেউ শুধু লাইফের হিসাব করে, কেউ শুধু দুটো বাত্ম ছোঁয়াছুঁয়ি হলো কিনা দেখে), আর **main.cpp** এই সবাইকে ঠিক সময়ে ডেকে ডেকে একটা পূর্ণ, চলমান খেলা বানিয়ে দেয়।

মনে রাখার মতো সবচেয়ে গুরুত্বপূর্ণ তিনটা ধারণা:

১. ভাগ করে কাজ করা — বড় কাজকে (পুরো খেলা বানানো) ছোট ছোট ফাংশনে ভাগ করলে বোঝা আর ঠিক করা অনেক সহজ হয়ে যায়।

২. কলব্যাক ও লুপ — `display()`, `keyboard()`, `update()` — এই তিনটে ফাংশন GLUT নিজে থেকে ডাকে, আর `update()` প্রতি ১৬ মিলিসেকেন্ডে নিজেকে আবার ডেকে একটা কখনো-না-খামা লুপ তৈরি করে, যেটাই খেলাটাকে জীবন্ত রাখে।

৩. পাবলিক বনাম গোপন — প্রতিটা ফাইল তার হেডারে (.h) শুধু যেটুকু বাইরে দরকার সেটুকুই দেখায়, বাকি সব হিসাব-নিকাশ নিজের ভেতরে গোপন (static) রাখে। এতে একটা ফাইলে কিছু বদলালে অন্য ফাইলগুলো ভেঙে পড়ে না।

স্টোন রানার (Dino_Sprint) প্রজেক্টের কোড পড়ে তৈরি করা একটা সহজ ব্যাখ্যা। কোনো কোড এখানে বদলানো হয়নি — শুধু বোঝানো হয়েছে।